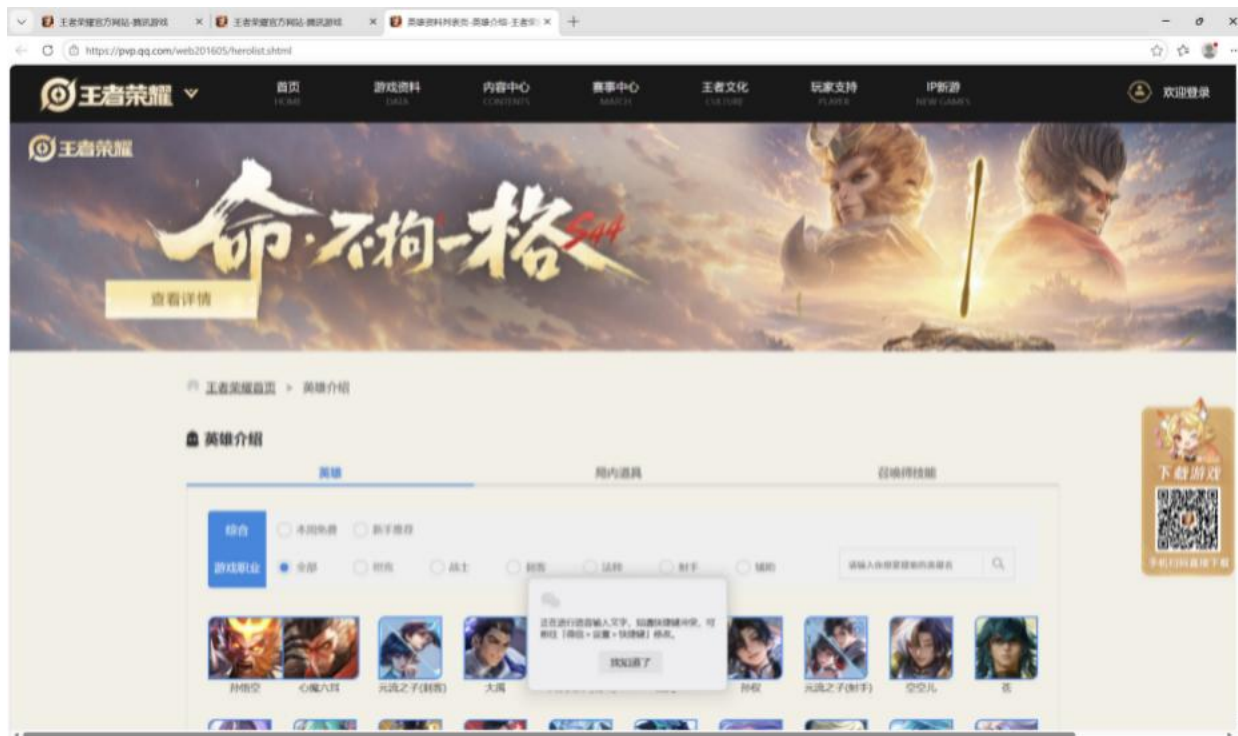


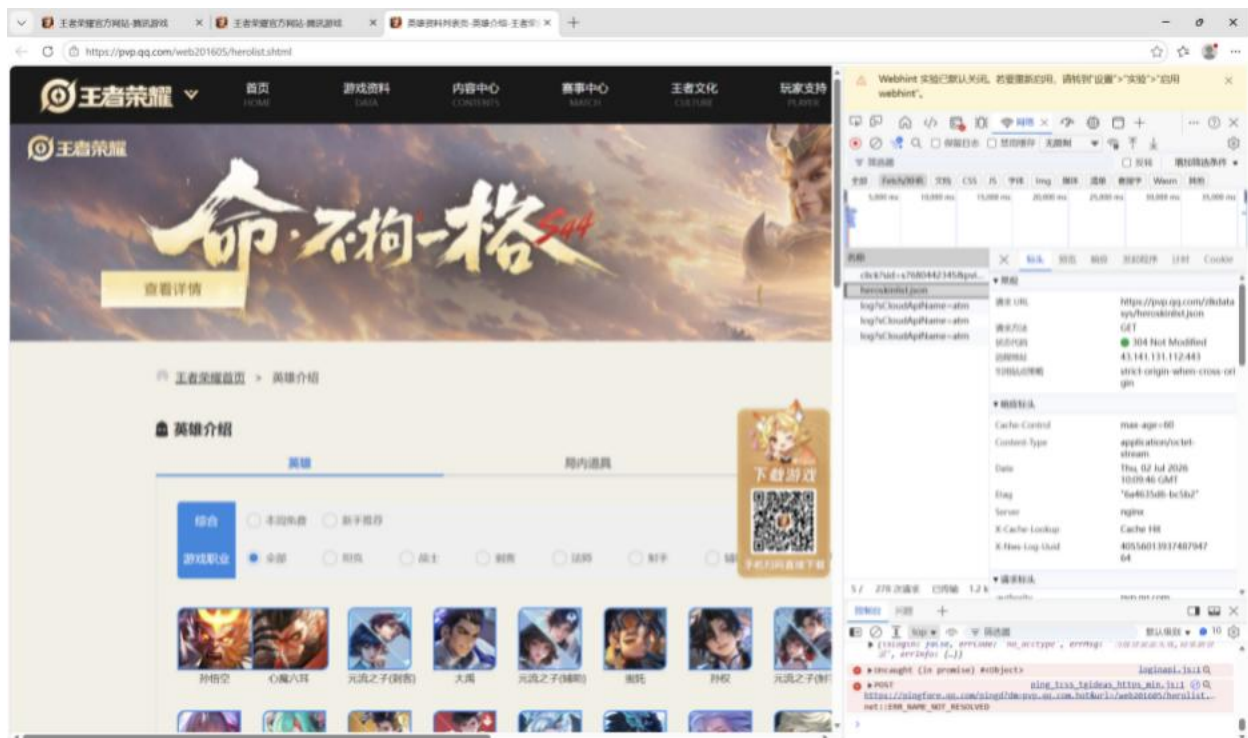
对英雄皮肤进行数据分析

本项目通过爬取腾讯王者荣耀公开数据分析热门游戏皮肤数据与数值，通过 `numpy`、`matplotlib` 对数据进行分析与可视化。

打开浏览器访问王者荣耀官方网站 (<https://pvp.qq.com/>)，进入顶部导航栏「游戏资料 - 英雄资料」板块，打开全英雄列表页面。该页面的皮肤数据为前端异步渲染加载，静态 HTML 源码不包含完整结构化数据，因此通过开发者工具抓取底层接口数据。



按下键盘 F12 键打开浏览器开发者工具，切换至 **Network（网络）** 面板，在筛选栏选择 Fetch/XHR 类型，过滤掉图片、样式文件等无关请求。刷新页面后，在请求列表中定位到 herolist.json 接口文件，点击查看响应内容，确认该接口返回全英雄的基础信息此文件为爬虫部分全部代码



一、爬虫部分：crawl_.py

1. 此文件为爬虫部分代码
2. 文件功能：爬取了官网英雄皮肤信息
3. 生成一个 csv 文件:王者荣耀皮肤数据.csv

	英雄名称 ▾	皮肤名称 ▾	皮肤品质 ▾	上线时间 ▾	获取方式 ▾
1	赵云	飞龙神将	传说	20260703	商城直售获取
2	姬小满	灵囍仙官	史诗	20260703	商城直售获取
3	孙权	仙游使者	史诗	20260625	商城直售获取
4	艾琳	繁夏菱	史诗	20260625	S44赛季战令进阶版获取
5	赵怀真	逢春序	勇者→史诗	20260625	S44赛季战令免费版获取
6	金蝉	济世神心	史诗	20260625	碎片商店兑换&商城直售获取
7	海诺	暗夜裁决	勇者	20260601	月度福利礼册&商城直售获取
8	墨子	碎光执事	勇者	20260529	限时活动&碎片商店兑换&商城直售获取
9	杨戬	神目破阵	勇者	20260522	60点券限时秒杀
10	朵莉亚	幻海寻迹	勇者	20260520	碎片商店兑换&商城直售获取
11	刘禅	超级夜猫侠	史诗	20260501	5月礼册上新
12	铠	东海龙王·敖光	珍品传说	20260429	限时活动获取
13	芈月	西海龙王·敖闰	珍品传说	20260429	限时活动获取
14	敖隐	灵珠·敖丙	传说限定	20260429	商城直售获取
15	哪吒	魔丸·哪吒	传说限定	20260429	商城直售获取
16	姜子牙	弘道录	史诗	20260429	限时活动&商城直售获取
17	鲁班七号	极速特派	勇者	20260423	限时活动&碎片商店&商城直售获取
18	米莱狄	青瓷雀吟	史诗	20260423	60点券限时秒杀
19	后羿	天穹守望者	史诗	20260401	S43赛季战令进阶版获取
20	海月	海之女神	史诗	20260401	S43赛季战令免费版获取
21	大乔	微风如诗	勇者	20260401	碎片商店兑换&商城直售获取
22	雅典娜	执弈定局	勇者	20260401	月度福利礼册&商城直售获取
23	彭	百相守梦	勇者	20260319	限时活动&碎片商店兑换&商城直售获取
24	阿珂	罗刹妄海	传说	20260306	商城直售获取
25	达摩	星君·亢金	史诗	20260227	商城直售获取
26	韩信	踏浪归	传说	20260220	商城直售获取
27	上官婉儿	春露将至	史诗	20260218	商城直售获取
28	貂蝉	神曦·紫万那	传说	20260215	商城限时直售获取
29	马超	神曦·逐星野	传说	20260215	商城限时直售获取
30	狄仁杰	神曦·赴明漠	传说	20260215	商城限时直售获取

```

import requests
import csv
import json

# 王者荣耀皮肤数据接口地址
url = "https://pvp.qq.com/zlkdatasys/heroskinlist.json"

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
    "Referer": "https://pvp.qq.com/"
}

# 1. 请求并解析原始 JSON
response = requests.get(url, headers=headers, timeout=10)
response.encoding = "utf-8"

```

```

full_data = response.json()

# 2. 提取真实的皮肤列表数组
skin_list = full_data["pf1b20_3469"]
print(f"接口实际返回皮肤数量: {len(skin_list)}")

# 3. 字段重命名映射 (英文原始字段 -> 中文表头)
field_map = {
    "pfid1b_3934": "皮肤 ID",
    "pfmclb_7523": "皮肤名称",
    "yxmclb_9965": "英雄名称",
    "pfpzlb_3289": "皮肤品质",
    "sxsjlb_1516": "上线时间",
    "yjhjsl_5003": "皮肤简介",
    "hqfs_8609": "获取方式",
    "yxtxlb_8443": "宣传大图 URL",
    "fmlb_4536": "封面图 URL",
    "fmb1lb_5300": "Banner 图 URL",
    "pcljlb_9272": "PC 详情页",
    "mdljlb_1924": "移动端详情页",
    "spvidl_6663": "视频 ID",
    "pfgift_4455": "动态封面 URL",
    "pfbqlb_1811": "分类编号"
}

# 4. 清洗数据, 转换为中文键名的字典列表
clean_data = []
for skin in skin_list:
    clean_skin = {}
    for en_key, cn_key in field_map.items():
        # 字段不存在时默认填充空字符串
        clean_skin[cn_key] = skin.get(en_key, "")
    clean_data.append(clean_skin)

# 5. 使用 csv 模块保存为 CSV 文件 (utf-8-sig 解决 Excel 中文乱码)
output_file = "王者荣耀皮肤数据.csv"
csv_headers = list(field_map.values()) # 提取中文表头

with open(output_file, "w", encoding="utf-8-sig", newline="") as f:
    # newline="" 避免 Windows 下出现多余空行
    writer = csv.DictWriter(f, fieldnames=csv_headers)

```

```
writer.writeheader()          # 写入表头行
writer.writerows(clean_data)   # 批量写入所有皮肤数据

print(f"数据已保存到 {output_file}, 共 {len(clean_data)} 条记录")
```

二、数据清洗部分: date_wash1.py date_wash2.py

1. 此文件为数据清洗全部代码
2. 主要功能为对原始爬虫数据做全量清洗, 补充空值, 截取前 156 行数据, 摘取需要的列
3. 最终生成 csv:王者荣耀皮肤数据_.csv

date_wash1.py

```
import csv
# 配置文件路径
input_file = "王者荣耀皮肤数据.csv" # 原文件
output_file = "王者荣耀皮肤数据_.csv" # 生成的新文件
keep_rows = 156 # 保留的总行数 (包含表头)

# 读取并截取前 N 行写入新文件
with open(input_file, "r", encoding="utf-8-sig", newline="") as f_in:
    reader = csv.reader(f_in)
    rows = []
    for idx, row in enumerate(reader):
        if idx >= keep_rows:
            break
        rows.append(row)

# 写入新文件
with open(output_file, "w", encoding="utf-8-sig", newline="") as f_out:
    writer = csv.writer(f_out)
    writer.writerows(rows)

print(f"处理完成, 已保留前 {len(rows)} 行数据, 新文件已保存为: {output_file}")
```

date_wash2.py

```
import csv
```

```
# 配置项
input_file = "王者荣耀皮肤数据_.csv" # 原文件路径
output_file = "王者荣耀皮肤数据_.csv" # 处理后新文件路径
# 需要保留的列（顺序即为最终输出的列顺序）
keep_columns = ["英雄名称", "皮肤名称", "皮肤品质", "上线时间", "获取方式"]
fill_value = "无官方数据" # 空值填充内容

# 统计变量
replace_count = 0
total_rows = 0

# 读取原文件 + 处理数据
processed_data = []
with open(input_file, "r", encoding="utf-8-sig", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        # 筛选指定列，按顺序提取
        new_row = {col: row[col].strip() for col in keep_columns}

        # 处理获取方式空值（空字符串、纯空格都判定为空）
        if new_row["获取方式"] == "":
            new_row["获取方式"] = fill_value
            replace_count += 1

        processed_data.append(new_row)
        total_rows += 1

# 写入新文件
with open(output_file, "w", encoding="utf-8-sig", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=keep_columns)
    writer.writeheader() # 写入表头
    writer.writerows(processed_data)

# 输出处理结果
print(f"处理完成，共处理 {total_rows} 条数据")
print(f"已保留列: {keep_columns}")
print(f"获取方式空值填充数量: {replace_count} 条")
print(f"新文件已保存为: {output_file}")
```

三、数据标准化部分：数据标准化.py

1. 字段拆分归一：把「传说限定」这类混合品质拆成「基础品质 + 是否限定」两个独立字段；把零散的获取描述，统一归为直售 / 抽奖 / 战令 / 免费福利 / 无官方数据 5 大类，统一统计口径
2. 时间标准化：校验日期合法性，拆分出「上线年份」「上线月份」两个独立维度，方便时间维度分析
3. 衍生标签：自动生成「付费层级」（高端 / 中端 / 入门 / 免费）等分析标签，后续分析无需重复计算
4. 最终得到：王者荣耀皮肤数据_标准化版..csv

```
import csv
from collections import Counter

INPUT_FILE = "王者荣耀皮肤数据_1.csv"          # 原始数据文件
OUTPUT_FILE = "王者荣耀皮肤数据_标准化版.csv"  # 处理后输出的新文件
# 可选：是否开启近两年数据过滤（默认关闭，需要筛选 2024-2025 年可改为 True）
ENABLE_YEAR_FILTER = False
START_YEAR = 2024
END_YEAR = 2025

# 获取方式分类匹配规则
GET_TYPE_RULES = [
    ("限定返场类", ["返场"]),
    ("抽奖类", ["夺宝", "祈愿", "抽奖", "积分", "荣耀典藏", "无双"]),
    ("战令类", ["战令"]),
    ("免费福利类", ["赛季", "免费", "活动", "碎片", "签到", "兑换", "任务"]),
    ("直售类", ["点券", "商城", "直售", "首周", "购买"])
]

# 付费层级对应规则
PAY_LEVEL_MAP = {
    "荣耀典藏": "高端付费",
    "无双": "高端付费",
    "珍品传说": "高端付费",
    "传说": "高端付费",
    "史诗": "中端付费",
    "勇者": "入门付费",
    "伴生": "入门付费"
}

# 存储处理后的数据
```

```

processed_data = []
# 统计变量
total_raw = 0
invalid_date_count = 0
replace_null_count = 0

# 1. 读取原始数据并逐行处理
with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        total_raw += 1
        clean_row = {
            "英雄名称": row["英雄名称"].strip(),
            "皮肤名称": row["皮肤名称"].strip(),
            "原始品质": row["皮肤品质"].strip(),
            "原始上线时间": row["上线时间"].strip(),
            "原始获取方式": row["获取方式"].strip()
        }
        # 获取方式空值填充
        if clean_row["原始获取方式"] == "":
            clean_row["原始获取方式"] = "无官方数据"
            replace_null_count += 1

        quality_raw = clean_row["原始品质"]
        is_limited = "是" if "限定" in quality_raw else "否"
        # 剔除“限定”字样，得到基础品质
        base_quality = quality_raw.replace("限定", "").strip()
        clean_row["基础品质"] = base_quality
        clean_row["是否限定"] = is_limited

        get_type_raw = clean_row["原始获取方式"]
        final_type = "无官方数据"
        for type_name, keywords in GET_TYPE_RULES:
            if any(keyword in get_type_raw for keyword in keywords):
                final_type = type_name
                break
        clean_row["获取方式分类"] = final_type

        date_str = clean_row["原始上线时间"]
        year = ""
        month = ""

```



```

        date_valid = "否"
        # 适配 8 位数字日期格式
        if len(date_str) == 8 and date_str.isdigit():
            year = date_str[:4]
            month = date_str[4:6]
            # 校验日期合理性
            if 1 <= int(month) <= 12 and 2015 <= int(year) <= 2030:
                date_valid = "是"
            else:
                invalid_date_count += 1
        else:
            invalid_date_count += 1
        clean_row["上线年份"] = year
        clean_row["上线月份"] = month
        clean_row["日期是否有效"] = date_valid

        clean_row["付费层级"] = PAY_LEVEL_MAP.get(base_quality, "免费/其他")

        if ENABLE_YEAR_FILTER and date_valid == "是":
            if not (START_YEAR <= int(year) <= END_YEAR):
                continue

        processed_data.append(clean_row)

# 2. 写入标准化后的新文件
output_columns = [
    "英雄名称", "皮肤名称",
    "基础品质", "是否限定", "原始品质",
    "获取方式分类", "原始获取方式",
    "上线年份", "上线月份", "日期是否有效", "原始上线时间",
    "付费层级"
]

with open(OUTPUT_FILE, "w", encoding="utf-8-sig", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=output_columns)
    writer.writeheader()
    writer.writerows(processed_data)

# 3. 打印处理结果汇总报告
print("=" * 50)
print("【数据标准化处理完成报告】")

```

```
print("=" * 50)
print(f"原始数据总条数: {total_raw}")
print(f"处理后有效数据条数: {len(processed_data)}")
print(f"获取方式空值填充数量: {replace_null_count} 条")
print(f"日期格式异常数量: {invalid_date_count} 条")
print(f"输出文件: {OUTPUT_FILE}")
print("-" * 50)
```

分类统计预览

```
quality_counter = Counter([row["基础品质"] for row in processed_data])
get_type_counter = Counter([row["获取方式分类"] for row in processed_data])
limited_count = sum(1 for row in processed_data if row["是否限定"] == "是")
```

```
print(f"【基础品质分布预览】")
```

```
for q, cnt in sorted(quality_counter.items(), key=lambda x:x[1],
reverse=True):
```

```
    print(f"    {q}: {cnt} 款")
```

```
print(f"\n【获取方式分类预览】")
```

```
for t, cnt in sorted(get_type_counter.items(), key=lambda x:x[1],
reverse=True):
```

```
    print(f"    {t}: {cnt} 款")
```

```
print(f"\n限定皮肤总数量: {limited_count} 款, 占比
{limited_count/len(processed_data)*100:.2f}%")
```

```
print("=" * 50)
```

最终数据形式如下:

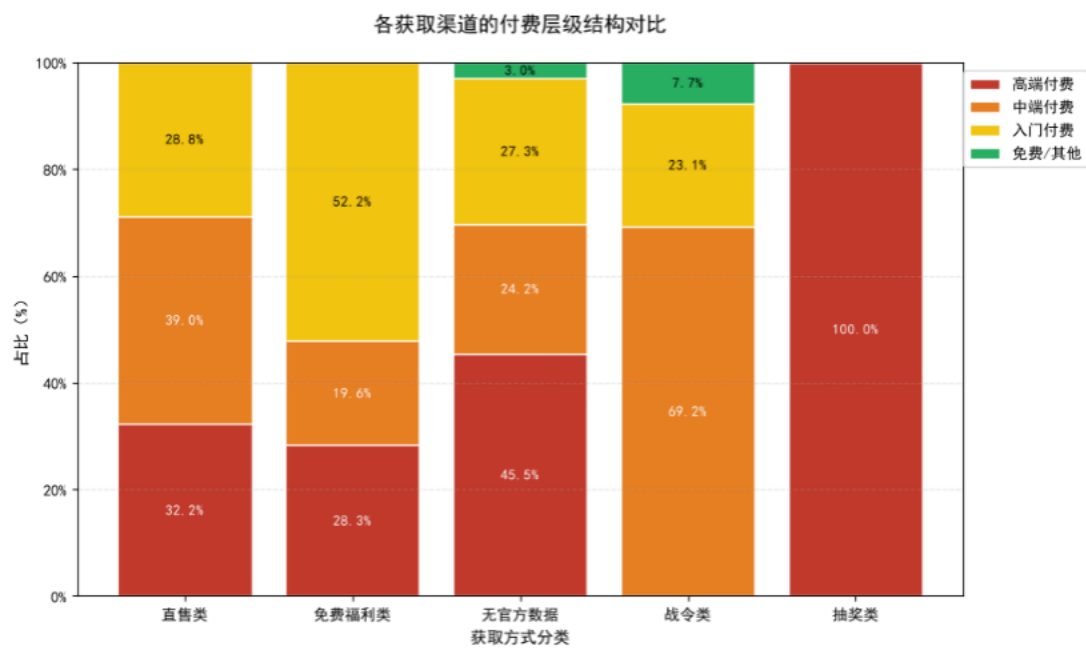
英雄名称	皮肤名称	基础品质	是否限定	原画品质	获取方式	原始获取时间年份	上线月份	日期是否	原始上线时付费情况
赵云	飞龙神将	传说	否	传说	直售类	商城直售	2026/07	是	20260703 高端付费
橘小虞	英雄仙侠	史诗	否	史诗	直售类	商城直售	2026/07	是	20260703 中端付费
孙权	仙鹿使者	史诗	否	史诗	直售类	商城直售	2026/06	是	20260625 中端付费
艾琳	誓翼瑞	史诗	否	史诗	战令类	543赛季战	2026/06	是	20260625 中端付费
赵怀真	逐香伴	勇者一史	否	勇者一史	战令类	544赛季战	2026/06	是	20260625 免费/其他
金蝉	济世神心	史诗	否	史诗	免费福利	碎片商店	2026/06	是	20260625 中端付费
海诺	辉夜裁决	勇者	否	勇者	直售类	月度福利	2026/06	是	20260601 入门付费
墨子	神光执事	勇者	否	勇者	免费福利	限时活动	2026/05	是	20260529 入门付费
杨戬	神日破阵	勇者	否	勇者	直售类	60点券限	2026/05	是	20260522 入门付费
朵莉亚	幻海寻迹	勇者	否	勇者	免费福利	碎片商店	2026/05	是	20260520 入门付费
刘备	超超夜猫	史诗	否	史诗	无官方数据	5月礼上	2026/05	是	20260501 中端付费
貂蝉	东海龙王	珍品传说	否	珍品传说	免费福利	限时活动	2026/04	是	20260429 高端付费
半鱼	西南龙王	珍品传说	否	珍品传说	免费福利	限时活动	2026/04	是	20260429 高端付费
敖隐	灵珠散月	传说	是	传说限定	直售类	商城直售	2026/04	是	20260429 高端付费
哪吒	魔丸哪吒	传说	是	传说限定	直售类	商城直售	2026/04	是	20260429 高端付费
姜子牙	弘道录	史诗	否	史诗	免费福利	限时活动	2026/04	是	20260429 中端付费
鲁班七号	极速特选	勇者	否	勇者	免费福利	限时活动	2026/04	是	20260423 入门付费
米莱狄	青瓷夜羽	史诗	否	史诗	直售类	60点券限	2026/04	是	20260423 中端付费
后羿	天穹守望	史诗	否	史诗	直售类	543赛季战	2026/04	是	20260401 中端付费
海月	海之女神	史诗	否	史诗	战令类	543赛季战	2026/04	是	20260401 中端付费
大乔	微风如诗	勇者	否	勇者	免费福利	碎片商店	2026/04	是	20260401 入门付费
雅典娜	扶桑定局	勇者	否	勇者	直售类	月度福利	2026/04	是	20260401 入门付费
影	百相守梦	勇者	否	勇者	免费福利	限时活动	2026/03	是	20260319 入门付费
阿珂	罗刹安海	传说	否	传说	直售类	商城直售	2026/03	是	20260306 高端付费
达摩	慈悲元象	史诗	否	史诗	直售类	商城直售	2026/02	是	20260227 中端付费
韩信	踏浪归	传说	否	传说	直售类	商城直售	2026/02	是	20260222 高端付费
上官婉儿	春露诗至	史诗	否	史诗	直售类	商城直售	2026/02	是	20260218 中端付费
貂蝉	神像万	传说	是	传说限定	直售类	商城限时	2026/02	是	20260215 高端付费
马超	神像速展	传说	是	传说限定	直售类	商城限时	2026/02	是	20260215 高端付费
狄仁杰	神像迅影	传说	是	传说限定	直售类	商城限时	2026/02	是	20260215 高端付费
朵莉亚	神像鸣幻	传说	是	传说限定	直售类	商城限时	2026/02	是	20260215 高端付费
关羽	神像镇古	传说	是	传说限定	直售类	商城限时	2026/02	是	20260215 高端付费
云缨	神像越时	传说	是	传说限定	直售类	商城限时	2026/02	是	20260215 高端付费
安琪拉	熊树童话	荣耀典藏	否	荣耀典藏	免费福利	荣耀物品	2026/02	是	20260215 高端付费
李信	同心剑	无双	否	无双	抽奖类	无双新愿	2026/02	是	20260215 高端付费
伽罗	沧浪游	珍品传说	否	珍品传说	免费福利	限时活动	2026/02	是	20260215 高端付费
明世隐	百晓客	史诗	否	史诗	免费福利	限时活动	2026/02	是	20260215 中端付费
李元芳	星程奏全	传说	是	传说限定	免费福利	参与奖励	2026/02	是	20260215 高端付费

四、数据分析与可视化分析部分

1. 获取方式与付费层级分析

按获取方式分类拆分，统计每类渠道里高 / 中 / 入门付费皮肤的占比。看清每个渠道的商业定位，比如抽奖类是不是几乎全是高端皮肤、直售的主力档位是不是史诗、免费福利是不是集中在入门品质，完整拆解官方的分层营收体系。

采用 matplotlib 中 plt.plot() 方法绘制百分比堆叠柱状图(代码见附录)

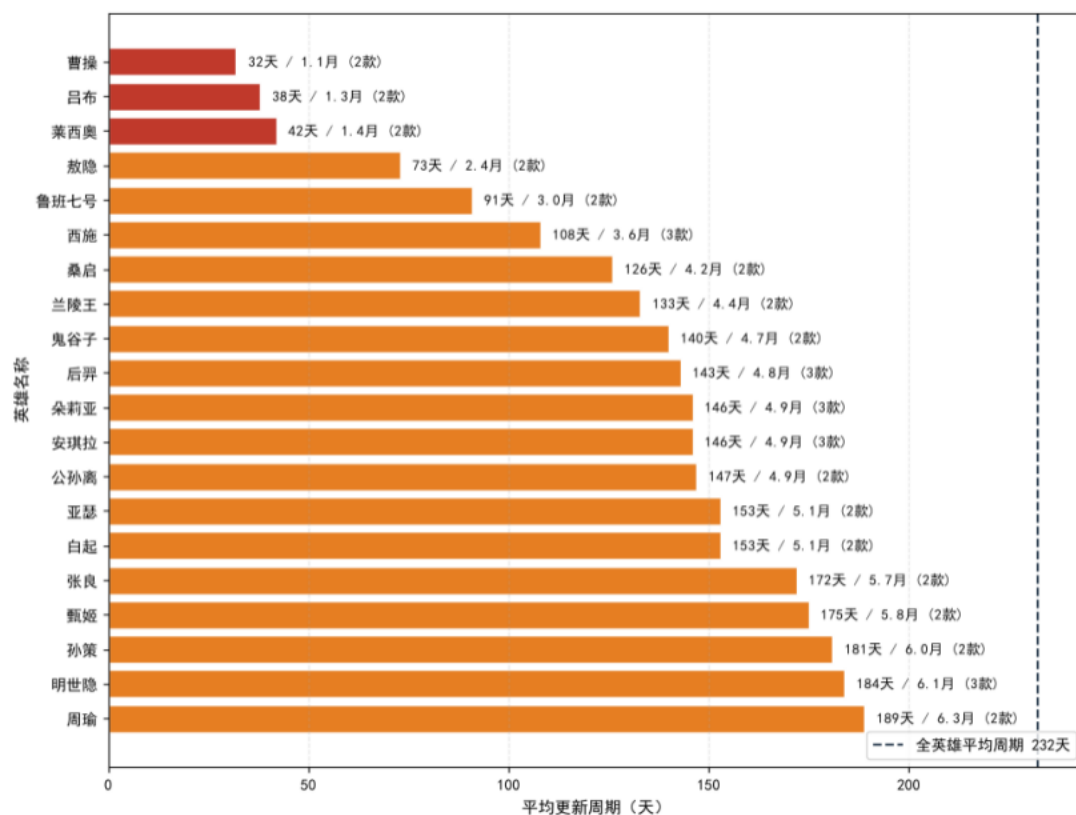


2. 英雄更新皮肤频率分析

针对有 2 款及以上新皮肤的英雄，计算相邻两款皮肤的上线间隔，得出每个英雄的平均更新周期，再做排序。补充 “皮肤总量” 之外的维度，识别出官方近期高频更新的重点英雄，有些英雄总皮肤不多但近两年更新极快，往往是当前的主推运营对象。

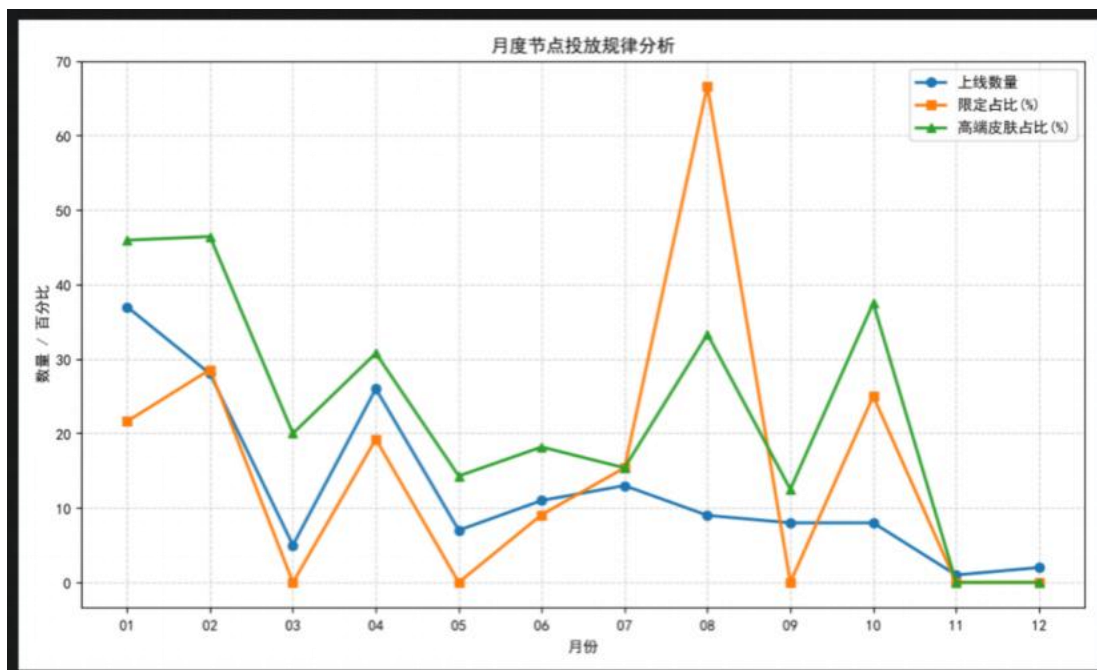
采用 matplotlib 中 plt.plot() 方法绘制 Top20 英雄更新频率横向条形图(代码见附录)

英雄皮肤更新频率 Top20（周期越短更新越快）



3. 月度节点投放规律分析

把时间维度从“年”细化到“月”，统计每月的上新数量，找出上线量峰值月份；再对应峰值月份，统计当月的限定占比、高端皮肤占比，匹配春节、暑期、周年庆等核心节点。采用 matplotlib 中 `plt.plot()` 方法绘制月度节点投放规律分析（代码见附录）



五、分析总结

1. 获取方式与付费层级分析结论

直售渠道以中端皮肤为核心，覆盖高中低不同消费层级，是最主要、最稳定的收入来源。免费福利渠道主要承担拉新、留存和节日活动功能，通过大量基础品质皮肤提升玩家活跃度，同时少量高品质奖励增强活动吸引力。

战令渠道聚焦中端品质，以较高的性价比促进赛季持续付费，形成稳定的长期收益模式。

抽奖渠道专注于高端品质皮肤，通过稀缺性和随机机制提升用户付费深度，是高价值商业化的重要组成部分。总体来看，王者荣耀并未采用单一的销售模式，而是根据不同获取方式配置不同品质和付费层级的皮肤，形成了“**”免费引流—直售转化—战令留存—抽奖高价值变现“**”的多层次商业化体系。这种分层运营既覆盖了不同消费能力的玩家群体，也提高了皮肤产品的生命周期价值和整体盈利能力，是游戏长期运营的重要支撑。

2. 英雄皮肤更新频率分析

统计 45 位拥有 2 款及以上皮肤的英雄，全英雄平均更新周期为 232 天（约 7.7 个月），周期越短代表官方运营优先级越高。

榜单梯队解读

超高频梯队（前 5，平均周期 < 3 个月）：曹操、吕布、莱西奥、敖隐、鲁班七号，以短期集中投放为主，对应节点高人气英雄与快速商业化的新英雄。

中高频梯队（6-15 名，3-6 个月）：西施、后羿、安琪拉等，为长期常态化核心运营梯队，覆盖多位置，更新节奏稳定。

常规高频梯队（16-20 名，6-7 个月）：张良、甄姬、孙策等，属次核心梯队，配合版本节点补充皮肤。

最终定位结论

长期 S 级主推：西施、后羿、安琪拉、朵莉亚皮肤数量多 + 更新周期短，商业价值最高

短期重点投放：曹操、吕布、鲁班七号节点集中资源倾斜，短期运营优先级高）

潜力上升英雄：敖隐、莱西奥（新英雄快速补皮，官方正在抬升其商业定位）

3. 月度节点投放规律分析

春节节点（1~2 月）

春节期间不仅皮肤上线数量较多，而且高端皮肤占比较高，说明官方会利用春节这一玩家活跃度最高的时期推出高价值皮肤，提升销售收入和节日活动影响力。

暑期节点（7~8 月）

暑期期间限定皮肤占比达到全年最高，说明官方重点通过限定皮肤吸引学生群体及暑期活跃玩家，利用限时销售策略增强消费意愿。

周年庆节点（10 月）

10 月高端皮肤和限定皮肤占比均保持较高水平，体现出周年庆作为年度重要运营节点，会集中推出高品质、限定类皮肤，进一步提高玩家参与度和品牌影响力。

普通月份

3 月、5 月、9 月等月份上线数量较少，限定及高端皮肤占比较低，说明这些月份主要承担版本更新和日常运营功能，而不是商业化重点投放阶段。

附录

一、获取方式与付费层级分析

```
import csv
from collections import defaultdict, Counter
```

```

import matplotlib.pyplot as plt
import numpy as np

file_path = "王者荣耀皮肤数据_标准化版.csv"
# 解决中文乱码问题
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'PingFang SC']
plt.rcParams['axes.unicode_minus'] = False

# 交叉统计：获取方式 × 付费层级
channel_pay = defaultdict(Counter)
with open(file_path, "r", encoding="utf-8-sig", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        if row["日期是否有效"] == "是":
            channel = row["获取方式分类"]
            pay_level = row["付费层级"]
            channel_pay[channel][pay_level] += 1

# 固定付费层级顺序
pay_levels = ["高端付费", "中端付费", "入门付费", "免费/其他"]
# 渠道按皮肤总数量降序排序，图表更规整
channels = sorted(channel_pay.keys(), key=lambda x:
sum(channel_pay[x].values()), reverse=True)

# 转换为百分比数据
percent_data = []
for level in pay_levels:
    level_percents = []
    for ch in channels:
        ch_total = sum(channel_pay[ch].values())
        cnt = channel_pay[ch].get(level, 0)
        percent = cnt / ch_total * 100 if ch_total > 0 else 0
        level_percents.append(percent)
    percent_data.append(level_percents)

plt.figure(figsize=(10, 6), dpi=100)

# 配色：从深到浅对应付费层级从高到低，辨识度高
colors = ["#C0392B", "#E67E22", "#F1C40F", "#27AE60"]

# 逐层绘制堆叠柱

```

```

bottom = np.zeros(len(channels))
for i in range(len(pay_levels)):
    bars = plt.bar(
        channels,
        percent_data[i],
        bottom=bottom,
        color=colors[i],
        label=pay_levels[i],
        edgecolor='white' # 柱子分段加白边，更清晰
    )
    # 分段内添加百分比标签
    for j in range(len(channels)):
        if percent_data[i][j] > 3:
            plt.text(
                j,
                bottom[j] + percent_data[i][j] / 2,
                f"{percent_data[i][j]:.1f}%",
                ha='center',
                va='center',
                fontsize=9,
                # 深色背景用白字，浅色背景用黑字，保证可读性
                color='white' if i < 2 else 'black'
            )
    bottom += percent_data[i]

plt.title("各获取渠道的付费层级结构对比", fontsize=14, pad=20)
plt.ylabel("占比 (%)", fontsize=11)
plt.xlabel("获取方式分类", fontsize=11)
plt.ylim(0, 100) # Y轴固定 0-100%，符合百分比堆叠逻辑
plt.yticks(range(0, 101, 20), [f"{x}%" for x in range(0, 101, 20)])
plt.legend(loc="upper right", bbox_to_anchor=(1.15, 1)) # 图例放外侧，不遮挡柱子
plt.grid(axis='y', linestyle='--', alpha=0.3) # 加横向网格线，方便读数
plt.tight_layout()

# 保存 300DPI 高清图+ 显示图表
plt.savefig("获取方式付费层级堆叠图.png", dpi=300, bbox_inches='tight')
plt.show()

```

二、英雄更新皮肤频率分析


```

import csv
from datetime import datetime
from collections import defaultdict
import matplotlib.pyplot as plt

file_path = "王者荣耀皮肤数据_标准化版.csv"
DATE_FORMAT = "%Y%m%d" # 适配 8 位数字日期格式
# 解决 matplotlib 中文乱码
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'PingFang SC']
plt.rcParams['axes.unicode_minus'] = False

#第一部分：数据计算
# 按英雄分组收集有效上线日期
hero_date_map = defaultdict(list)
with open(file_path, "r", encoding="utf-8-sig", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        if row["日期是否有效"] == "是":
            hero_name = row["英雄名称"].strip()
            date_str = row["原始上线时间"].strip()
            try:
                skin_date = datetime.strptime(date_str, DATE_FORMAT).date()
                hero_date_map[hero_name].append(skin_date)
            except ValueError:
                continue

# 计算每个英雄的更新周期指标
update_cycle_result = []
for hero, date_list in hero_date_map.items():
    skin_total = len(date_list)
    if skin_total < 2:
        continue
    date_sorted = sorted(date_list)
    interval_days = [(date_sorted[i] - date_sorted[i-1]).days for i in
range(1, len(date_sorted))]
    update_cycle_result.append({
        "英雄名称": hero,
        "皮肤总数": skin_total,
        "平均周期(天)": round(sum(interval_days) / len(interval_days)),
        "平均周期(月)": round(sum(interval_days) / len(interval_days) / 30,
1),

```

```

        "最短间隔(天)": min(interval_days),
        "最长间隔(天)": max(interval_days)
    })

# 按平均周期升序排序
update_cycle_result.sort(key=lambda x: x["平均周期(天)"])
# 全英雄平均周期
all_avg_days = sum([x["平均周期(天)"] for x in update_cycle_result]) /
len(update_cycle_result)

#第二部分：控制台输出统计榜单
print("=" * 80)
print("【英雄皮肤更新频率排行榜 Top20】（周期越短，更新频率越高）")
print("=" * 80)
print(f"{'排名':<4}{'英雄名称':<12}{'皮肤总数':<8}{'平均周期(天)':<14}{'平均周'
期(月)':<14}{'最短间隔(天)':<12}")
print("-" * 80)
for rank, item in enumerate(update_cycle_result[:20], 1):
    print(f"{rank:<6}{item['英雄名称']:<14}{item['皮肤总数']:<10}{item['平均周'
期(天)':<16}{item['平均周期(月)':<16}{item['最短间隔(天)':<12}")
print("-" * 80)
print(f"统计范围：共 {len(update_cycle_result)} 位英雄拥有 2 款及以上皮肤")
print(f"全英雄平均更新周期：{round(all_avg_days)} 天，约
{round(all_avg_days/30,1)} 个月")
print("=" * 80)

# 取 Top20 数据并反转（让第一名显示在图表最上方）
top20 = update_cycle_result[:20][::-1]
hero_names = [x["英雄名称"] for x in top20]
skin_counts = [x["皮肤总数"] for x in top20]
avg_days = [x["平均周期(天)"] for x in top20]
avg_months = [x["平均周期(月)"] for x in top20]

plt.figure(figsize=(10, 8), dpi=100)
# 配色：Top3 高亮红色，其余统一橙色
colors = ["#E67E22"] * 17 + ["#C0392B"] * 3
bars = plt.barh(hero_names, avg_days, color=colors, edgecolor='white')

# 添加全英雄平均基准线
plt.axvline(
    x=all_avg_days,

```

```

        color="#2C3E50",
        linestyle="--",
        linewidth=1.5,
        label=f"全英雄平均周期 {round(all_avg_days)}天"
    )

# 柱子末端标注数值
for bar, days, months, count in zip(bars, avg_days, avg_months, skin_counts):
    plt.text(
        bar.get_width() + 3,
        bar.get_y() + bar.get_height()/2,
        f"{days}天 / {months}月 ({count}款)",
        va='center',
        fontsize=9
    )

# 图表美化
plt.title("英雄皮肤更新频率 Top20 (周期越短更新越快)", fontsize=14, pad=20)
plt.xlabel("平均更新周期 (天)", fontsize=11)
plt.ylabel("英雄名称", fontsize=11)
plt.legend(loc="lower right")
plt.grid(axis='x', linestyle='--', alpha=0.3)
plt.tight_layout()

# 保存高清图 + 显示图表
plt.savefig("英雄更新频率排行.png", dpi=300, bbox_inches='tight')
plt.show()

```

三、月度节点投放规律分析

```

import csv
import matplotlib.pyplot as plt

# 解决 matplotlib 中文显示乱码问题
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 存储所有标准化后的皮肤数据
data = []

```

```

with open("王者荣耀皮肤数据_标准化版(1).csv", "r", encoding="utf-8-sig") as f:
    # 以字典形式读取 CSV，每一行对应一个皮肤的完整字段
    reader = csv.DictReader(f)
    for row in reader:
        data.append(row)

# 打印皮肤总数量，校验数据读取是否正常
print("皮肤总数：", len(data))

def statistics(column):
    """
    统计指定列的各取值出现次数
    :param column: 要统计的列名（如“基础品质”、“上线年份”）
    :return: 字典，格式为 {字段值: 出现次数}
    """
    result = {}

    for row in data:
        value = row[column]

        if value in result:
            result[value] += 1
        else:
            result[value] = 1

    return result

def draw_bar(dic, title, xlabel, ylabel):
    """
    绘制柱状图
    :param dic: 统计字典 {类别: 数量}
    :param title: 图表标题
    :param xlabel: X 轴标签
    :param ylabel: Y 轴标签
    """
    # 拆分 X 轴类别和对应数值
    names = list(dic.keys())
    counts = list(dic.values())

    # 设置画布尺寸
    plt.figure(figsize=(8, 5))

```

```

# 绘制柱状图
plt.bar(names, counts)
# 设置标题和坐标轴标签
plt.title(title)
plt.xlabel(xlabel)
plt.ylabel(ylabel)

# 在每个柱子顶部添加数值标签
for i, value in enumerate(counts):
    plt.text(i, value + 0.5, str(value), ha='center')

# 自动调整布局，避免标签被截断
plt.tight_layout()
plt.show()

def draw_pie(dic, title):
    """
    绘制饼图（展示占比）
    :param dic: 统计字典 {类别: 数量}
    :param title: 图表标题
    """
    names = list(dic.keys())
    counts = list(dic.values())

    plt.figure(figsize=(7, 7))
    # 绘制饼图，autopct 自动显示百分比
    plt.pie(
        counts,
        labels=names,
        autopct="%1.1f%%",
        startangle=90 # 饼图起始角度
    )
    plt.title(title)
    plt.show()

def draw_line(dic, title):
    """
    绘制折线图（适合时间趋势类数据）
    :param dic: 统计字典 {年份: 数量}

```

```

:param title: 图表标题
"""

# 年份按升序排序，保证趋势线顺序正确
years = sorted(dic.keys())
counts = []
for y in years:
    counts.append(dic[y])

plt.figure(figsize=(8, 5))
# 绘制折线，marker='o' 给每个数据点加圆点标记
plt.plot(
    years,
    counts,
    marker='o'
)
plt.title(title)
plt.xlabel("年份")
plt.ylabel("皮肤数量")
# 添加网格线方便读数
plt.grid()
plt.show()

# 5.1 基础品质分布分析
quality = statistics("基础品质")
print("\n 基础品质统计: ")
for k, v in quality.items():
    print(k, v)

# 柱状图展示各品质数量
draw_bar(
    quality,
    "基础品质分布",
    "基础品质",
    "数量"
)

# 饼图展示各品质占比
draw_pie(
    quality,
    "基础品质占比"
)

```

5.2 限定皮肤占比分析

```
limited = statistics("是否限定")
print("\n 是否限定统计: ")
for k, v in limited.items():
    print(k, v)
```

```
draw_pie(
    limited,
    "限定皮肤占比"
)
```

5.3 获取方式分类分析

```
method = statistics("获取方式分类")
print("\n 获取方式分类统计: ")
for k, v in method.items():
    print(k, v)
```

```
draw_bar(
    method,
    "获取方式分类统计",
    "获取方式",
    "数量"
)
```

5.4 付费层级分析

```
pay = statistics("付费层级")
print("\n 付费层级统计: ")
for k, v in pay.items():
    print(k, v)
```

```
draw_bar(
    pay,
    "付费层级统计",
    "付费层级",
    "数量"
)
```

5.5 年度上线趋势分析

```
year = statistics("上线年份")
print("\n 上线年份统计: ")
for k, v in year.items():
```

```

    print(k, v)

draw_line(
    year,
    "上线年份变化趋势"
)

# 初始化三个字典：分别存储每个月的「总上线数量」「限定皮肤数量」「高端皮肤数量」
month = {}
limited = {}
high = {}

# 定义高端皮肤的品质范围
high_quality = [
    "传说",
    "珍品传说",
    "荣耀典藏",
    "无双"
]

# 遍历所有皮肤，按月度累加统计
for row in data:
    month_now = row["上线月份"]

    # 如果是该月第一条数据，初始化字典值为0
    if month_now not in month:
        month[month_now] = 0
        limited[month_now] = 0
        high[month_now] = 0

    # 总数量+1
    month[month_now] += 1
    # 限定皮肤+1
    if row["是否限定"] == "是":
        limited[month_now] += 1
    # 高端皮肤+1
    if row["基础品质"] in high_quality:
        high[month_now] += 1

```



```

# 打印月度统计明细：总上线数、限定占比、高端占比
print("\n 月份分析")
months = sorted(month.keys(), key=int) # 月份按数字升序排序
for m in months:
    total = month[m]
    limit_percent = limited[m] / total * 100
    high_percent = high[m] / total * 100
    print(
        f"{m}月  "
        f"上线:{total}  "
        f"限定:{limit_percent:.1f}%  "
        f"高端:{high_percent:.1f}%"
    )

# 绘制多折线对比图：同时展示月度上线量、限定占比、高端占比
months = sorted(month.keys(), key=int)
online_num = []
limit_rate = []
high_rate = []

for m in months:
    online_num.append(month[m])
    # 计算限定、高端皮肤的月度占比
    limit_rate.append(limited[m] / month[m] * 100)
    high_rate.append(high[m] / month[m] * 100)

plt.figure(figsize=(10, 6))
# 三条折线分别对应三个指标
plt.plot(months, online_num, marker='o', linewidth=2, label="上线数量")
plt.plot(months, limit_rate, marker='s', linewidth=2, label="限定占比(%)")
plt.plot(months, high_rate, marker='^', linewidth=2, label="高端皮肤占比(%)")

plt.title("月度节点投放规律分析")
plt.xlabel("月份")
plt.ylabel("数量 / 百分比")
plt.xticks(months)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend() # 显示图例
plt.tight_layout()
plt.show()

```